Asignatura:

Programación de Sistemas Avanzados

Sección D01

Profesor:

Jorge Ernesto Lopez Arce Delgado

Reporte 1

Entrega de avances

Equipo “Rust In Peace (RIN)”

Integrantes:

Ascencio Palma César Noé.

Delgadillo Becerra Andrea.

Guzmán Cabrera Álvaro.

Simental Beaven Oleta Nora Esthela.

Fecha de entrega: 19 de Febrero 2026

Índice

1. Introducción **3**
2. Diseño de la Arquitectura VPN. **3**
2.1 Justificación Técnica de la Topología 3
2.2 Diagrama Preliminar 3
2.3 Descripción de cada nodo 3
3.2 Generación de Llaves 4
4. Validación de Conectividad **4**
4.1 Verificación handshake entre los 4 nodos desde el HUB **4**
4.2 Verificación de pings hacia todos los nodos desde Peer (Peer 3) 5
5. Preparación del Entorno de Contenedores **5**
5.1 Instalación de Docker y docker-compose 5
5.2 Archivo docker-compose.yml Base 6
5.3 Archivo Dockerfile Base 6
7. Problemas Técnicos y Decisiones Adoptadas **6**
8. Definición del Problema Computacional Distribuido **7**
9. Rust **8**
9.1 Estructura del proyecto **8**
9.2 Definición de mensajes / estructuras 8
9.3 Tarea dummy 9
9.4 Qué falta implementar 10
9.5 Qué se probará primero 10
9.6 Cómo se medirá el rendimiento 11
9.7 Prueba de tolerancia a fallas 11
10. Github **11**

1. Introducción

El presente documento expone el avance correspondiente a la primera fase del proyecto, cuyo propósito es el diseño e implementación de una infraestructura base para la ejecución de un sistema distribuido.

En esta etapa se configuró una red privada virtual (VPN) utilizando WireGuard bajo una topología tipo hub-and-spoke, garantizando comunicación segura entre nodos ubicados en distintas máquinas virtuales Linux. Asimismo, se preparó un entorno de contenedores Docker en cada nodo y se validó su conectividad sobre la red privada creada. Finalmente, se definió el problema computacional distribuido que será desarrollado en Rust en la siguiente fase del proyecto.

2. Diseño de la Arquitectura VPN.

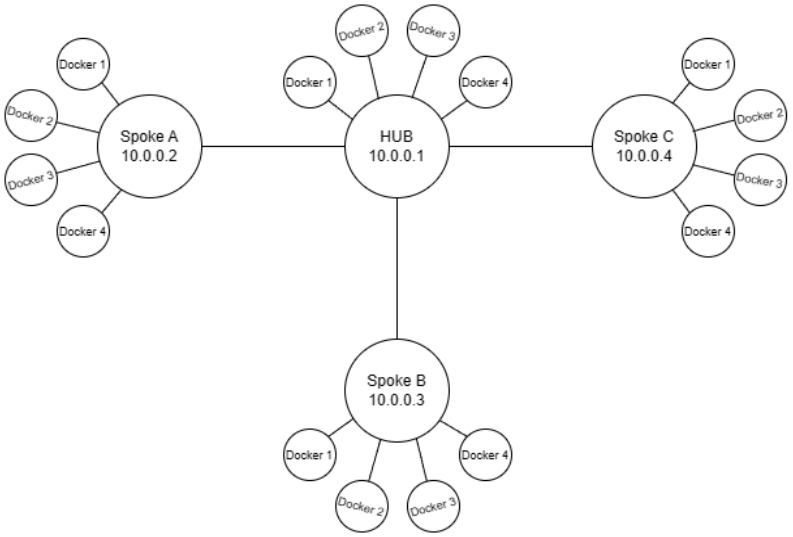
2.1 Justificación Técnica de la Topología

Se adoptó una arquitectura hub-and-spoke, donde un nodo central (Hub) actúa como punto de interconexión para todos los nodos cliente (Peers).

Esta arquitectura fue seleccionada debido a:

- Simplificación del ruteo.
- Centralización de la administración.
- Facilidad para implementar un modelo coordinador–workers en fases posteriores.

2.2 Diagrama Preliminar



2.3 Descripción de cada nodo

Hub: Dispositivo central de la infraestructura.

- Tiene la interfaz principal de WireGuard y mantiene las llaves públicas de los 3 peers.

- Permite y redirige el paso de paquetes entre túneles para que se puedan comunicar los peers entre ellos.
- Ejecuta 4 contenedores Docker.

Spoke A, B y C: Peers / spokes, nodos de sucursal.

- Buscan la IP del hub para establecer el túnel.
- Ejecutan 4 contenedores Dockers.

Contenedores Docker: End-hosts.

- Ejecutan la lógica del algoritmo distribuido de Rust.

3. Implementación de la VPN con WireGuard.

3.2 Generación de Llaves

```
alvarogc@hub-server: ~/proy
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ wg genkey | tee privatekey | wg pubkey > publickey
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ ls -l
total 8
-rw-rw-r-- 1 alvarogc alvarogc 45 Feb 15 03:34 privatekey
-rw-rw-r-- 1 alvarogc alvarogc 45 Feb 15 03:34 publickey
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ cat publickey
5zkHZoWZT1Wo5WZI5NwA18xwl9dfR2pb6QXWxPxDVSA=
alvarogc@hub-server:~/proyecto-rin/vpn/keys$
```

4. Validación de Conectividad

4.1 Verificación handshake entre los 4 nodos desde el HUB

```
ivang@LAPTOP-KCVLMDHQ:
[Nodo-Andrea] ~ $ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.2/24 dev wg0
[#] ip link set mtu 1392 up dev wg0
[#] resolvconf -a wg0 -m 0 -x
[Nodo-Andrea] ~ $ sudo wg show
interface: wg0
  public key: m61y9qWq/2t2iJ34AJlitbY1BJOn1SY2o1g/nfkWY1o=
  private key: (hidden)
  listening port: 50498

peer: 5zkHZoWZT1Wo5WZI5NwA18xwl9dfR2pb6QXWxPxDVSA=
  endpoint: 192.168.100.186:51820
  allowed ips: 10.0.0.0/24
  latest handshake: 29 seconds ago
  transfer: 92 B received, 212 B sent
  persistent keepalive: every 25 seconds
[Nodo-Andrea] ~ $
```

```
mario-ubuntu@ubuntu-serve
[Nodo-Nora] ~ $ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.4/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[Nodo-Nora] ~ $ sudo wg show
interface: wg0
  public key: NWgnYISSv4oWytP5T3/xjPm3NT47hs6PJm2915u7ME0=
  private key: (hidden)
  listening port: 46924

peer: 5zkHZoWZT1Wo5WZI5NwA18xwl9dfR2pb6QXWxPxDVSA=
  endpoint: 192.168.100.186:51820
  allowed ips: 10.0.0.0/24
  latest handshake: 23 seconds ago
  transfer: 92 B received, 180 B sent
  persistent keepalive: every 25 seconds
[Nodo-Nora] ~ $
```

```
alvarogc@hub-server: ~
[Nodo-Alvaro] ~ $ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
[Nodo-Alvaro] ~ $ sudo wg show
interface: wg0
  public key: 5zkHZoWZT1Wo5WZI5NwA18xwl9dfR2pb6QXWxPxDVSA=
  private key: (hidden)
  listening port: 51820

peer: AbfVXAPwyGJ87zyW4cQISezzMMJUCSw6JZ4vSXzkKXY=
  endpoint: 192.168.100.188:43493
  allowed ips: 10.0.0.3/32
  latest handshake: 37 seconds ago
  transfer: 212 B received, 92 B sent

peer: m61y9qWq/2t2iJ34AJlitbY1BJOn1SY2o1g/nfkWY1o=
  endpoint: 192.168.100.96:50498
  allowed ips: 10.0.0.2/32
  latest handshake: 46 seconds ago
  transfer: 212 B received, 92 B sent

peer: NWgnYISSv4oWytP5T3/xjPm3NT47hs6PJm2915u7ME0=
  endpoint: 192.168.100.196:46924
  allowed ips: 10.0.0.4/32
  latest handshake: 51 seconds ago
  transfer: 212 B received, 92 B sent
[Nodo-Alvaro] ~ $
```

```
guia — guia@ubuntu-server: ~ — ssh guia@192.168.100.188 — 120×30
[Nodo-Palma] ~ $ sudo wg-quick up wg0
[sudo] password for guia:
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.3/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] resolvconf -a wg0 -m 0 -x
[Nodo-Palma] ~ $ sudo wg show
interface: wg0
  public key: AbfVXAPwyGJ87zyW4cQISezzMMJUCSw6JZ4vSXzkKXY=
  private key: (hidden)
  listening port: 43493

peer: 5zkHZoWZT1Wo5WZI5NwA18xwl9dfR2pb6QXWxPxDVSA=
  endpoint: 192.168.100.186:51820
  allowed ips: 10.0.0.0/24
  latest handshake: 26 seconds ago
  transfer: 92 B received, 212 B sent
  persistent keepalive: every 25 seconds
[Nodo-Palma] ~ $
```

4.2 Verificación de pings hacia todos los nodos desde Peer (Peer 3)

```
guia — guia@ubuntu-server: ~ — ssh guia@192.168.100.188 — 120×30
[Nodo-Palma] ~ $ ping -c 4 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=12.3 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=12.6 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=7.20 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=10.4 ms

--- 10.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3012ms
rtt min/avg/max/mdev = 7.202/10.619/12.623/2.154 ms
[Nodo-Palma] ~ $ ping -c 4 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=63 time=18.3 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=63 time=16.6 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=63 time=17.3 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=63 time=22.3 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3016ms
rtt min/avg/max/mdev = 16.635/18.619/22.264/2.183 ms
[Nodo-Palma] ~ $ ping -c 4 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=63 time=15.4 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=63 time=32.5 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=63 time=13.5 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=63 time=14.6 ms

--- 10.0.0.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 13.478/19.027/32.549/7.837 ms
```

```
ivang@LAPTOP-KCVLMDHQ:
[Nodo-Andrea] ~ $
[Nodo-Andrea] ~ $ ping -c 4 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=5.00 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=5.60 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=5.84 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=5.36 ms

--- 10.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 4.995/5.447/5.838/0.311 ms
[Nodo-Andrea] ~ $ ping -c 4 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.142 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.048 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.040 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.057 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.040/0.071/0.142/0.041 ms
[Nodo-Andrea] ~ $ ping -c 4 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=63 time=12.5 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=63 time=11.0 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=63 time=11.0 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=63 time=10.9 ms

--- 10.0.0.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3023ms
rtt min/avg/max/mdev = 10.876/11.342/12.495/0.667 ms
[Nodo-Andrea] ~ $
```

```
mario-ubuntu@ubuntu-serve
[Nodo-Nora] ~ $ ping -c 4 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=8.79 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=6.76 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=7.31 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=5.87 ms

--- 10.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 5.870/7.182/8.785/1.058 ms
[Nodo-Nora] ~ $ ping -c 4 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=63 time=64.7 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=63 time=86.8 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=63 time=111 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=63 time=30.7 ms

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 30.669/73.299/110.977/29.549 ms
[Nodo-Nora] ~ $ ping -c 4 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=0.033 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.056 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.031 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=0.044 ms

--- 10.0.0.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3064ms
rtt min/avg/max/mdev = 0.031/0.041/0.056/0.010 ms
[Nodo-Nora] ~ $
```

```
alvarogc@hub-server: ~
[Nodo-Alvaro] ~ $
[Nodo-Alvaro] ~ $
[Nodo-Alvaro] ~ $ ping -c 4 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=5.46 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=6.60 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=5.15 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=5.45 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 5.152/5.666/6.604/0.555 ms
[Nodo-Alvaro] ~ $ ping -c 4 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=78.9 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=104 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=7.45 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=45.5 ms

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 7.450/58.884/103.709/36.168 ms
[Nodo-Alvaro] ~ $ ping -c 4 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=6.53 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=6.48 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=6.87 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=6.02 ms

--- 10.0.0.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 6.016/6.473/6.865/0.302 ms
[Nodo-Alvaro] ~ $
```

5. Preparación del Entorno de Contenedores

5.1 Instalación de Docker y docker-compose

```
alvarogc@hub-server: ~/proy   Windows PowerShell
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [40.2 kB]
Hit:4 http://mx.archive.ubuntu.com/ubuntu noble InRelease
Hit:5 http://mx.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://mx.archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 92.7 kB in 1s (92.8 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin doc
ker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras libslirp0 pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin libslirp0 pigz
  slirp4netns
0 upgraded, 9 newly installed, 0 to remove and 0 not upgraded.
Need to get 96.7 MB of archives.
After this operation, 391 MB of additional disk space will be used.
Get:1 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 2.2.1-1~ubuntu.24.04~noble [23.4 MB]
Get:2 http://mx.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:29.2.1-1~ubuntu.24.04~noble [16.3 MB]
Get:4 http://mx.archive.ubuntu.com/ubuntu noble/main amd64 libslirp0 amd64 4.7.0-1ubuntu3 [63.8 kB]
Get:5 http://mx.archive.ubuntu.com/ubuntu noble/universe amd64 slirp4netns amd64 1.2.1-1build2 [34.9 kB]
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:29.2.1-1~ubuntu.24.04~noble [22.5 MB]
```

```
alvarogc@hub-server: ~/proy   Windows PowerShell
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ docker compose version
Docker Compose version v5.0.2
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ sudo usermod -aG docker $USER
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ newgrp docker
alvarogc@hub-server:~/proyecto-rin/vpn/keys$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:ef54e839ef541993b4e87f25e752f7c4f4238fa55f017957c2eb44077083d7a6a
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

alvarogc@hub-server:~/proyecto-rin/vpn/keys$
```

5.2 Archivo docker-compose.yml Base

Code Blame 33 lines (29 loc) · 671 Bytes

```yaml
services:
  worker1:
    build:
      context: ../
      dockerfile: docker/Dockerfile
    container_name: worker1
    network_mode: host
    restart: unless-stopped

  worker2:
    build:
      context: ../
      dockerfile: docker/Dockerfile
    container_name: worker2
    network_mode: host
    restart: unless-stopped

  worker3:
    build:
      context: ../
      dockerfile: docker/Dockerfile
    container_name: worker3
    network_mode: host
    restart: unless-stopped

  worker4:
    build:
      context: ../
      dockerfile: docker/Dockerfile
    container_name: worker4
    network_mode: host
    restart: unless-stopped

```

5.3 Archivo Dockerfile Base

Code Blame 11 lines (9 loc) · 346 Bytes

```dockerfile
FROM rust:latest AS builder
WORKDIR /app
COPY rust/ ./rust/
WORKDIR /app/rust
RUN cargo build --release --bin worker

FROM ubuntu:24.04
RUN apt update && apt install -y libssl-dev ca-certificates curl iputils-ping && rm -rf /var/lib/apt/lists/*
COPY --from=builder /app/rust/target/release/worker /usr/local/bin/worker

CMD ["worker"]
```

7. Problemas Técnicos y Decisiones Adoptadas

CGNAT: El primer problema con el que nos encontramos fue el ir descubriendo, uno por uno, que no teníamos IP pública, hasta que llegamos a nuestro compañero César Palma, que su red no está detrás de un CGNAT como la del resto de nosotros.

WSL2: Tanto en la VPN local que montó Álvaro Guzmán, como en la que estaba creando César Palma con su IP pública, tuvimos problemas al crear el hub con la consola de Ubuntu en el Windows Subsystem for Linux, ya que WSL2 introduce capas de abstracción que complican la conectividad, ya que, según aprendimos:

- WSL2 no funciona con un puente de red por defecto, sino que utiliza un switch virtual Hyper-V con NAT.
- El kernel de WSL2 no incluye módulos de kernel por defecto necesarios o carece de soporte.
- Cada vez que se reinicia Windows o el servicio de WSL2, la IP interna de Ubuntu cambia.

- Windows bloquea por seguridad el tráfico de entrada que intenta ser redirigido hacia una interfaz virtual como WSL2.

Por todo esto, decidimos crear una máquina virtual dedicada en Hyper-V con una distribución de Ubuntu server, para configurar la tarjeta de red en modo Bridge, haciendo que la VM tenga su propia IP en la red local, eliminando la capa NAT y permitiendo un control total sobre el Kernel.

Problemas de Firewall y reglas de red:
Durante las pruebas de conexión, aún con el servicio de wireguard abierto tuvimos problemas para hacer handshake entre los peers. Teniendo en cuenta que teníamos el servicio bien configurado nos generó dudas en si las claves estaban mal copiadas o si había errores en el archivos wg0.conf.

Tras pensar y analizar más a fondo, el problema estaba relacionado con la configuración del firewall y el reenvío de paquetes. Nos dimos cuenta que no solo teníamos que tener el servicio activo, sino que también era necesario que el puerto UPD correspondiente estuviera realmente abierto, que las reglas de firewall permitieran tráfico entrante y que el sistema tuviera habilitado el IP forwarding para permitir el paso de los paquetes entre interfaces.

Este problema nos demostró que no solo es montar una VPN generando claves y escribiendo configuraciones, sino que también implicaba comprender cómo interactúan el SO, el firewall y la pila de red. El error fue asumir que el servicio funcionando garantizaba una conectividad exitosa.

Gestión ineficiente de claves públicas
Otro inconveniente práctico fue el proceso manual de copiar y pegar las claves públicas largas en el archivo wg0.conf. Las claves criptográficas al ser cadenas extensas y sensibles a errores mínimos tuvo a César Palma copiando uno por uno carácter por carácter y validando varias veces entre computadoras para validar que estuviera bien escrita.

Este error fue completamente fallo e ineficiencia humana ya que no se utilizó el SSH porque no fue contemplado a la hora de conectar las computadora principal con la computadora con Ubuntu Server nativo. La solución fue entablar una conexión SSH entre ambas computadoras para facilitar la modificación de las claves y cualquier edición de documento necesaria para que el programa funcionara correctamente.

Hyper-V: Es un hipervisor nativo de tipo 1 desarrollado por Microsoft, a diferencia de los tipo 2 como VMB que se ejecutan sobre el sistema operativo, Hyper-V se ejecuta directamente sobre el hardware del equipo. Nos encontramos con que no está disponible en la versión Home de Windows 11, por lo que nuestro compañero César Palma quedó en la imposibilidad de usarlo para crear el hub. Sin embargo, Álvaro Guzmán pudo usarlo para el hub en su red local.

8. Definición del Problema Computacional Distribuido

El problema computacional seleccionado para la siguiente fase del proyecto es la generación distribuida del Conjunto de Mandelbrot. Esta elección responde a su elevada

intensidad computacional, ya que el cálculo requiere ejecutar millones de iteraciones sobre puntos del plano complejo para construir una imagen de alta resolución.
Asimismo, cada unidad de cálculo es completamente independiente de las demás, lo que le confiere un paralelismo natural y lo convierte en un caso ideal para su ejecución en un entorno distribuido.
En el modelo propuesto, el plano complejo será dividido en franjas horizontales que el Coordinador mantendrá en una cola de tareas, de la cual los Workers tomarán cada una para procesar su bloque de manera autónoma y devolverá los resultados parciales, los cuales serán ensamblados por el Coordinador para generar la imagen final.

9. Rust

9.1 Estructura del proyecto

rust/Cargo.toml

Manifiesto del proyecto que contiene los metadatos del paquete (nombre, versión y edición de Rust), la declaración de dependencias externas y la configuración de perfiles de compilación.

rust/src/lib.rs

Representa el root create de una librería. El código aquí es público para otros creates del mismo paquete o para usuarios externos si se publica. Define la interfaz de programación (API) y organiza los módulos internos del proyecto.

rust/bin/worker.rs

Create binario que por estar separado genera un ejecutable independiente llamado “worker”. Importa la lógica de lib.rs para evitar duplicación de código.

rust/bin/coordinator.rs

Otro create binario, este tiene la lógica de ejecución de tareas.

9.2 Definición de mensajes / estructuras

```shell
use serde::{Deserialize, Serialize};

// Esto es lo que el Coordinador envía al Worker
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct Task {
    pub task_id: u32,
```

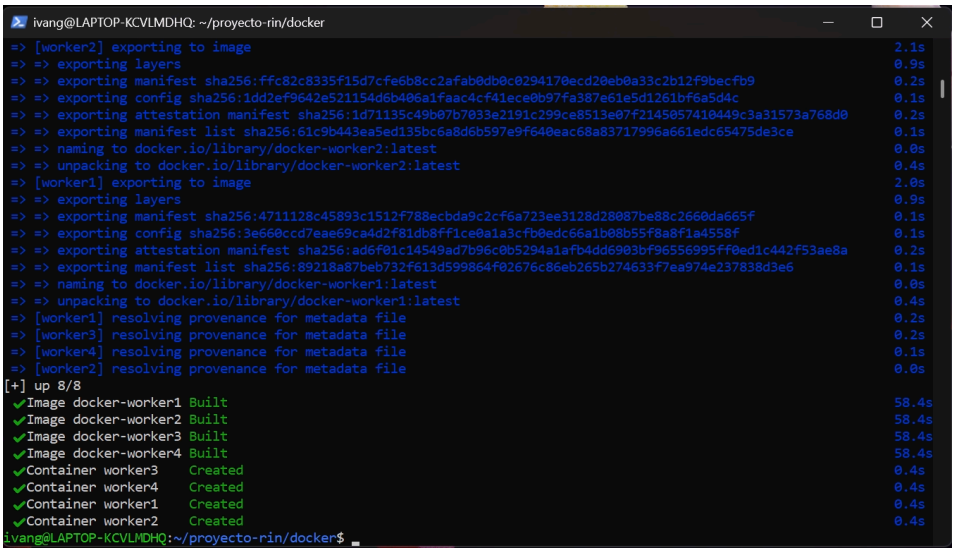
```rust
    pub instruction: String,
}

// Esto es lo que el Worker le responde al Coordinador
#[derive(Serialize, Deserialize, Debug)]
pub struct ResultPayload {
    pub task_id: u32,
    pub worker_id: String,
    pub data: String,
}
```

Task:
Define el trabajo pendiente. Su propósito es contener la información mínima necesaria para que un worker sepa qué debe calcular. Lleva un número de identificación (para no confundir una tarea con otra) y las instrucciones textuales o parámetros técnicos del cálculo que se requiere realizar.

ResultPayload:
Define el trabajo terminado. Es el paquete de datos que el worker envía de vuelta al Coordinador una vez que se ha procesado la instrucción. Su propósito es informar al Coordinador que la tarea ha sido finalizada y entregarle los datos resultantes. Incluye el identificador de la tarea original (para que el Coordinador sepa a qué encargo corresponde este resultado), la identidad del worker que lo hizo (para control y registro) y los datos de cálculo ya procesados.

9.3 Tarea dummy



```
alvarogc@hub-server: ~/proy
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Enviando tarea: Task { task_id: 1, instruction: "Calcula una tarea du
mmy de prueba" }
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
Resultado recibido del worker 'worker-01': "Cálculo completado para t
area 1"
```

9.4 Qué falta implementar

- **Integración del algoritmo de Mandelbrot:** Implementar en el worker la lógica de escape de iteraciones.
- **Dinamismo en los parámetros:** El coordinador deje de enviar tareas dummy y segmentar el plano complejo en reuniones o líneas de escaneo. El worker debe ser capaz de interpretar estas coordenadas específicas para renderizar solo su fracción asignada.
- **Escalabilidad horizontal:** Configurar el entorno para que múltiples instancias del worker ejecutándose en diferentes direcciones IP dentro de la red privada, puedan registrarse y solicitar tareas de forma concurrente sin generar condiciones de carrera en el coordinador.

9.5 Qué se probará primero

- **Validación del pipeline sobre wireguard:** Confirmar que el tráfico HTTP entre nodos no es bloqueado por las interfaces de la VPN y que la latencia introducida por el cifrado de wireguard es despreciable para el volumen de datos manejado.

- **Pruebas de carga de serialización:** Verificar que los mensajes JSON de gran tamaño se serialicen y deserialicen correctamente sin exceder los límites de memoria o los tiempos de espera.

9.6 Cómo se medirá el rendimiento

- **Benchmarking temporal:** Se utilizará la estructura std::time::Instant en el coordinador para capturar el tiempo exacto desd eque se despacha la primera tarea hasta que se recibe el último ResultPayload.
- **Análisis de escalabilidad**: Pruebas incrementales (1, 2, 3 y 4 nodos) para calcular el speedup del sistema. Esto permitirá identificar el punto de saturación donde el costo de comunicación empieza a restarle beneficios a la capacidad de cómputo agregada.

9.7 Prueba de tolerancia a fallas

- **Simulación de caída de nodo:** Se forzará la interrupción de un proceso worker en mitad de un cálculo para observar el comportamiento del sistema.
- **Mecanismo de reasignación:** Se validará que el coordinador implemente un sistema de heartbeats o un temporizador para tareas pendientes. Si una tarea no es reportada como terminada tras un tiempo, el coordinador debe devolverle a la cola de tareas pendientes para que otro worker disponible la procese, garantizando que la imagen final no tenga huecos.

10. Github

El proyecto se encuentra versionado en un repositorio GitHub, el cual centraliza la infraestructura, configuración y documentación del sistema distribuido implementado.El repositorio fue estructurado de manera modular con el objetivo de mantener claridad organizativa y facilitar la reproducibilidad del entorno.

La organización principal es la siguiente:

- **/vpn/configuraciones_sanitizadas**: Contiene las plantillas de configuración de WireGuard sin llaves privadas, garantizando seguridad.

- **/docker**: Incluye el archivo docker-compose.yml y los Dockerfile necesarios para la construcción y despliegue de los contenedores worker.

- **/rust**: Contiene la estructura base del proyecto en Rust, incluyendo el código del coordinador y los workers.

- **/docs/reportes**: Documentación técnica y entregables parciales.

- **README.md**: Instrucciones generales para instalación, configuración y ejecución del sistema.

LINK: https://github.com/andrea-debe/rustinpeace